

Laboratory Manual
For

Machine Learning
U24AIMLPC508

Semester – V

Artificial Intelligence
Machine Learning

List of Experiments and Assessment

Sr. No.	Name of Experiments	Date	Teacher Sign
1	Write a Program to find the mean, median, standard deviation and mode using user defined functions.		
2	Create a data frame with columns at least 5 observations. a) Retrieve a particular column from the DataFrame b) Summarize the data frame and observe the statistics of the DataFrame created c) Observe the mean and standard deviation of the data frame and print the values.		
3	Write a program to implement the Linear Regression for a sample training data set stored as a .CSV file. Compute Mean Square Error by considering few test data sets.		
4	Write a program to implement the Non-linear Regression for a sample training data set stored as a .CSV file. Compute Mean Square Error by considering few test data sets.		
5	Write a program to implement the Logistic Regression for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier.		
6	Write a program to implement the naive Bayesian classifier for a sample training data set stored as a .CSV file. Compute the accuracy of the classifier, considering few test data sets.		
7	Write a program to implement k-Nearest Neighbor algorithm to classify the Iris data set. Print both correct and wrong predictions.		
8	Write a program to demonstrate the working of the decision tree based ID3 algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.		
9	Write a program to demonstrate the working of the decision tree based CART algorithm. Use an appropriate data set for building the decision tree and apply this knowledge to classify a new sample.		

EXPERIMENT NO. 1

Title: Write a Program to find the mean, median, standard deviation and mode using user defined functions.

Aim

To write a Python program that computes the mean, median, standard deviation, and mode of a numeric dataset using user-defined functions, without relying on built-in statistics libraries.

Objectives

- Implement standard descriptive-statistics measures from first principles.
- Distinguish between measures of central tendency (mean, median, mode) and measures of dispersion (standard deviation).
- Handle both odd- and even-length datasets when computing the median.
- Handle multi-modal datasets when computing the mode.

Theory

Descriptive statistics summarise the main features of a dataset. The mean is the arithmetic average of all values. The median is the middle value of a sorted dataset; for an even number of observations it is the average of the two central values. The standard deviation measures the spread of data around the mean and is computed as the square root of the variance, the average of the squared deviations from the mean. The mode is the value (or values) that occur most frequently in the dataset; a dataset can have more than one mode when several values share the highest frequency.

Concepts Used

- Central tendency (mean, median, mode)
- Dispersion (variance and standard deviation)
- Sorting for median computation
- Frequency counting using Counter

System / Software Requirements

- Python 3.x
- Standard library module: collections
- Text editor / IDE

Algorithm

1. Accept a list of numeric values from the user.

2. Compute the mean by dividing the sum of values by the count of values.

3. Sort the data and compute the median from the middle element(s).

4. Compute the variance as the average squared deviation from the mean, then take its square root to get the standard deviation.

5. Use a frequency counter to find the value(s) with the highest occurrence count as the mode.

6. Print all four computed statistics.

Source Code

descriptive_stats.py

```python
from collections import Counter

def mean(data):
    return sum(data) / len(data)

def median(data):
    s = sorted(data)
    n = len(s)
    if n % 2 == 1:
        return s[n // 2]
    return (s[n // 2 - 1] + s[n // 2]) / 2

def standard_deviation(data):
    m = mean(data)
    variance = sum((x - m) ** 2 for x in data) / len(data)
    return variance ** 0.5

def mode(data):
    counts = Counter(data)
    max_count = max(counts.values())
    return [x for x, count in counts.items() if count == max_count]

data = list(map(float, input("Enter numbers separated by space: ").split()))

print("Mean:", mean(data))
print("Median:", median(data))
print("Standard Deviation:", standard_deviation(data))
print("Mode:", mode(data))
```

Execution

Command

```
python3 descriptive_stats.py
```

Sample Input

Input

```
Enter numbers separated by space: 12 15 12 18 20 15 12 22
```

Sample Output

Output

```
Mean: 15.75
Median: 15.0
Standard Deviation: 3.5637...
Mode: [12.0]
```

Test Cases

Case	Input / Condition	Expected Result
Odd count	5 7 9	median = 7
Even count	4 6 8 10	median = 7.0
Single mode	1 2 2 3	mode = [2]
Multi-modal	1 1 2 2 3	mode = [1, 2]

Result

Thus, a Python program was implemented to compute the mean, median, standard deviation, and mode of a dataset using user-defined functions.

EXPERIMENT NO. 2

Title: Create a data frame with columns at least 5 observations. a) Retrieve a particular column from the DataFrame b) Summarize the data frame and observe the statistics of the DataFrame created c) Observe the mean and standard deviation of the data frame and print the values.

Aim

To create a pandas DataFrame with at least five observations and demonstrate column retrieval, statistical summarisation, and computation of mean and standard deviation.

Objectives

- Construct a DataFrame from a Python dictionary of columns.
- Retrieve an individual column as a pandas Series.
- Use describe() to obtain summary statistics of numeric columns.
- Compute the mean and standard deviation of numeric columns directly.

Theory

A pandas DataFrame is a two-dimensional, labelled tabular data structure with columns that may hold different data types. It is the primary data structure used for data analysis in Python. Individual columns can be accessed like dictionary keys and returned as a Series. The describe() method computes count, mean, standard deviation, minimum, quartiles, and maximum for every numeric column in a single call, which is useful for a quick exploratory overview of a dataset before further analysis.

Concepts Used

- DataFrame construction
- Column indexing
- Descriptive summary via describe()
- Column-wise mean and standard deviation

System / Software Requirements

- Python 3.x
- pandas library
- Text editor / IDE

Algorithm

1. Define a dictionary containing at least five observations across several columns.

2. Construct a DataFrame from the dictionary.

3. Retrieve and display a chosen column using DataFrame indexing.

4. Call describe() to display summary statistics of the DataFrame.

5. Compute and display the mean and standard deviation of the numeric columns.

Source Code

dataframe_summary.py

```
import pandas as pd

data = {
    "Name": ["Asha", "Bharat", "Charan", "Divya", "Esha"],
    "Age": [20, 21, 19, 22, 20],
    "Marks": [78, 85, 69, 92, 88]
}

df = pd.DataFrame(data)

print("DataFrame:\n", df)
print("\nMarks column:\n", df["Marks"])
print("\nSummary Statistics:\n", df.describe())
print("\nMean:\n", df.mean(numeric_only=True))
print("\nStandard Deviation:\n", df.std(numeric_only=True))
```

Execution

Command

```
python3 dataframe_summary.py
```

Sample Input

Input

```
(No user input – dataset is defined in code)
```

Sample Output

Output

```
DataFrame:
     Name  Age  Marks
0    Asha   20     78
1  Bharat   21     85
...
Marks column: 0  78 ... 4  88
Summary Statistics: count, mean, std, min, 25%, 50%, 75%, max for Age and Marks
Mean:  Age  20.4  Marks  82.4
Standard Deviation:  Age  1.14  Marks  9.15
```

Test Cases

Case	Input / Condition	Expected Result
Column retrieval	df["Marks"]	returns Marks Series of length 5
Summary	df.describe()	returns 8-row statistics table
Mean	df.mean(numeric_only=True)	returns per-column mean

Case	Input / Condition	Expected Result
Std deviation	df.std(numeric_only=True)	returns per-column std

Result

Thus, a pandas DataFrame was created and column retrieval, statistical summarisation, and mean/standard-deviation computation were demonstrated.

EXPERIMENT NO. 3

Title: Write a program to implement the Linear Regression for a sample training data set stored as a .CSV file. Compute Mean Square Error by considering few test data sets.

Aim

To implement Linear Regression on a training dataset stored as a CSV file and evaluate the model using Mean Squared Error on a held-out test dataset.

Objectives

- Persist training and test data as CSV files and read them back with pandas.
- Fit a simple linear regression model of the form Y = mX + c.
- Generate predictions for unseen test inputs.
- Evaluate prediction quality using Mean Squared Error (MSE).

Theory

Linear Regression models the relationship between a dependent variable Y and an independent variable X as a straight line, Y = mX + c, where m is the slope and c is the intercept. The model parameters are learned by minimising the sum of squared residuals between predicted and actual values (ordinary least squares). Once fitted, the model can predict Y for new values of X. Mean Squared Error, the average of the squared differences between predicted and actual test values, is a standard metric for regression accuracy — lower MSE indicates a better fit.

Concepts Used

- Simple linear regression
- Ordinary least squares
- Train/test split via separate CSV files
- Mean Squared Error

System / Software Requirements

- Python 3.x
- pandas, scikit-learn libraries
- Text editor / IDE

Algorithm

1. Create training and test data and save each as a .CSV file.

2. Read the CSV files back into DataFrames.

3. Separate features (X) and target (Y) for both training and test sets.

4. Fit a LinearRegression model on the training data.

5. Predict Y for the test set inputs.

6. Compute and print the Mean Squared Error between predicted and actual test values.

Source Code

linear_regression.py

```
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

train = pd.DataFrame({
    "X": [1, 2, 3, 4, 5, 6, 7, 8],
    "Y": [3, 5, 7, 9, 11, 13, 15, 17]
})
test = pd.DataFrame({
    "X": [9, 10, 11],
    "Y": [19, 21, 23]
})

train.to_csv("linear_train.csv", index=False)
test.to_csv("linear_test.csv", index=False)

train_data = pd.read_csv("linear_train.csv")
test_data = pd.read_csv("linear_test.csv")

X_train = train_data[["X"]]
y_train = train_data["Y"]
X_test = test_data[["X"]]
y_test = test_data["Y"]

model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)

mse = mean_squared_error(y_test, predictions)
print("Predictions:", predictions)
print("Mean Squared Error:", mse)
```

Execution

Command
python3 linear_regression.py

Sample Input

Input
linear_train.csv: X=1..8, Y=2X+1 | linear_test.csv: X=9,10,11

Sample Output

Output
Predictions: [19. 21. 23.] Mean Squared Error: 0.0

Test Cases

Case	Input / Condition	Expected Result
Perfectly linear data	Y = 2X + 1	MSE ≈ 0
Held-out test points	X = 9,10,11	predictions close to actual Y
Model type	LinearRegression()	fits without error on numeric CSV

Result

Thus, a Linear Regression model was implemented using a CSV-based training and test dataset, and its accuracy was evaluated using Mean Squared Error.

EXPERIMENT NO. 4

Title: Write a program to implement the Non-linear Regression for a sample training data set stored as a .CSV file. Compute Mean Square Error by considering few test data sets.

Aim

To implement Non-linear (Polynomial) Regression on a training dataset stored as a CSV file and evaluate the model using Mean Squared Error on a test dataset.

Objectives

- Transform a single feature into polynomial features of a chosen degree.
- Fit a linear model over the expanded polynomial feature space to capture non-linear relationships.
- Generate predictions for unseen test inputs.
- Evaluate prediction quality using Mean Squared Error.

Theory

Many real-world relationships are not linear. Polynomial regression addresses this by expanding a single feature X into higher-degree terms (X, X^2, X^3, ...) and fitting an ordinary linear regression over this expanded feature space. Although the model is still linear in its parameters, the resulting curve, $Y = a + bX + cX^2 + ...$, can fit non-linear (curved) data. The degree of the polynomial controls the flexibility of the fit; scikit-learn's PolynomialFeatures generates the expanded feature matrix automatically.

Concepts Used

- Polynomial feature expansion
- Non-linear curve fitting via linear regression
- Degree selection
- Mean Squared Error

System / Software Requirements

- Python 3.x
- pandas, scikit-learn libraries
- Text editor / IDE

Algorithm

1. Create training and test data reflecting a quadratic relationship and save each as a .CSV file.

2. Read the CSV files back into DataFrames.

3. Expand the X feature into polynomial terms of degree 2 using PolynomialFeatures.

4. Fit a LinearRegression model on the expanded training features.

5. Transform the test features the same way and predict Y.

6. Compute and print the Mean Squared Error between predicted and actual test values.

Source Code

nonlinear_regression.py

```
import pandas as pd
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error

train = pd.DataFrame({
    "X": [1, 2, 3, 4, 5, 6, 7, 8],
    "Y": [3, 6, 11, 18, 27, 38, 51, 66]
})
test = pd.DataFrame({
    "X": [9, 10, 11],
    "Y": [83, 102, 123]
})

train.to_csv("nonlinear_train.csv", index=False)
test.to_csv("nonlinear_test.csv", index=False)

train_data = pd.read_csv("nonlinear_train.csv")
test_data = pd.read_csv("nonlinear_test.csv")

poly = PolynomialFeatures(degree=2, include_bias=False)
X_train = poly.fit_transform(train_data[["X"]])
X_test = poly.transform(test_data[["X"]])

model = LinearRegression()
model.fit(X_train, train_data["Y"])
predictions = model.predict(X_test)

mse = mean_squared_error(test_data["Y"], predictions)
print("Predictions:", predictions)
print("Mean Squared Error:", mse)
```

Execution

Command

```
python3 nonlinear_regression.py
```

Sample Input

Input

```
nonlinear_train.csv: X=1..8, Y=X²+X+1  |  nonlinear_test.csv: X=9,10,11
```

Sample Output

Output

```
Predictions: [ 83. 102. 123.]
Mean Squared Error: ~0.0
```

Test Cases

Case	Input / Condition	Expected Result
Quadratic data	$Y = X^2 + X + 1$	MSE ≈ 0 with degree=2
Degree too low	degree=1 on quadratic data	noticeably higher MSE
Held-out points	X = 9,10,11	predictions close to actual Y

Result

Thus, a Non-linear (Polynomial) Regression model was implemented using a CSV-based training and test dataset, and its accuracy was evaluated using Mean Squared Error.